

Q-1

How to find the mid node in LinkedList?

→

```
class Node {
```

```
public:
```

```
int data;
```

```
Node* next;
```

```
};
```

```
int getLen(class Node* head)
```

```
{
```

```
int len = 0;
```

```
class Node* temp = head;
```

```
while (temp) {
```

```
len++;
```

```
temp = temp->next;
```

```
return len;
```

```
}
```

```
void printMiddle(class Node* head)
```

```
{
```

```
if (head) {
```

```
int len = getLen(head);
```

```
class Node* temp = head;
```

```
int midInd = len/2;
```

```
while (midInd--)
```

```
temp = temp->next;
```

```
{
```

```
cout << temp->data << endl;
```



Wb.
because:

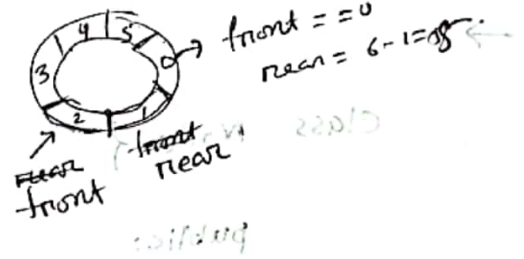
2 → 4 → 5 → 6

here, 5 is
middle node

Ques-02

Circular Queue

limit of wolf



```
void enqueue(int value) {
```

```
    if ((front == 0 && rear == size-1) ||  
        rear == (size-1)) {
```

```
        cout << "Queue is full" << endl;
```

```
    } else if (front == -1) { // insert 1st element
```

```
        front = rear = 0;
```

```
        arr[rear] = value;
```

```
    }
```

```
    else if (rear == size-1 && front != 0) {
```

```
        rear = 0;
```

```
        arr[rear] = value;
```

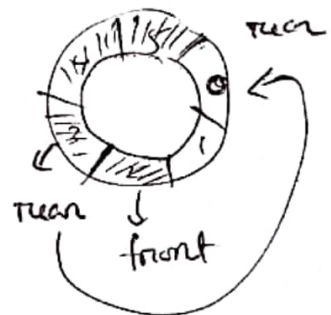
```
    }
```

```
    else {
```

```
        rear++;
```

```
        arr[rear] = value;
```

```
    }
```



```
int dequeue() {
```

```
    if (front == -1) {
```

```
        cout << "Queue is Empty" << endl;
```

```
    }
```

```
    int data = arr[front];
```

```
    arr[front] = -1; // initialize as -1
```

```
    if (front == rear) { // one element
```

```
        front = -1;
```

```
        rear = -1;
```

```
    }
```

```
    else if (front == size - 1) {
```

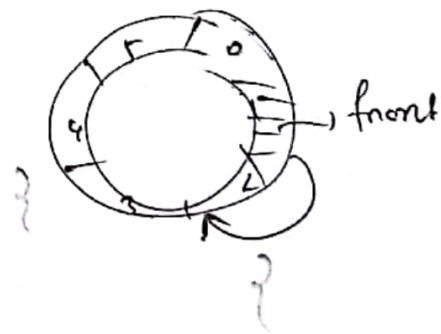
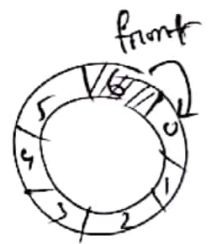
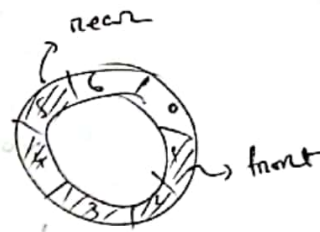
```
        front = 0;
```

```
    }
```

```
    else
```

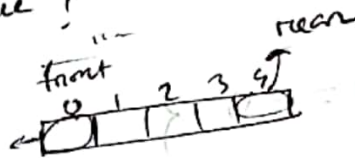
```
        front++;
```

```
    return data;
```

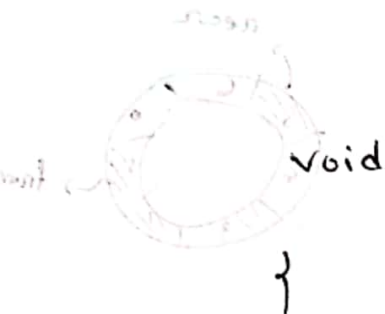


Q-3

How to reverse a queue?



→



void reverseQueue (queue <int> & Queue)

{
stack <int> stack;

while (! Queue.empty())

{
stack.push (Queue.front());

Queue.pop();

}

while (! stack.empty())

{
Queue.push (stack.pop());

stack.pop();

}

}



using recursion

front end

void reverseQueue (queue <int> &q)

{

if (q.size() == 0)

return;

int fn = q.front();

q.pop();

reverseQueue(q);

q.push(fn);

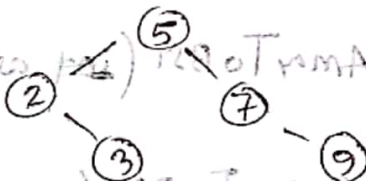
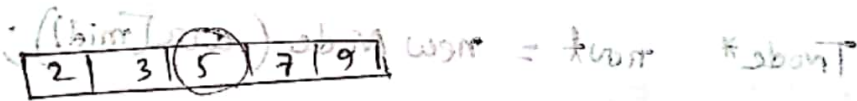
}



fn = 1
fn = 2
fn = 3
fn = 4



How to create BST (from an Sorted Array)?



return root;

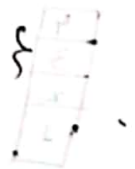
```
class Tnode {
```

```
public:
```

```
int data;
```

```
Tnode* left;
```

```
Tnode* right;
```



```
Tnode*
```

```
SortedArrayToBST (int arr[], int start, int end)
```

```
{
```

```
if (start > end)
```

```
return NULL;
```

```
int mid = (start + end) / 2;
```

```
Tnode* root = new Node(arr[mid]);
```

```
root->left = SortedArrayToBST(arr, start, mid-1);
```

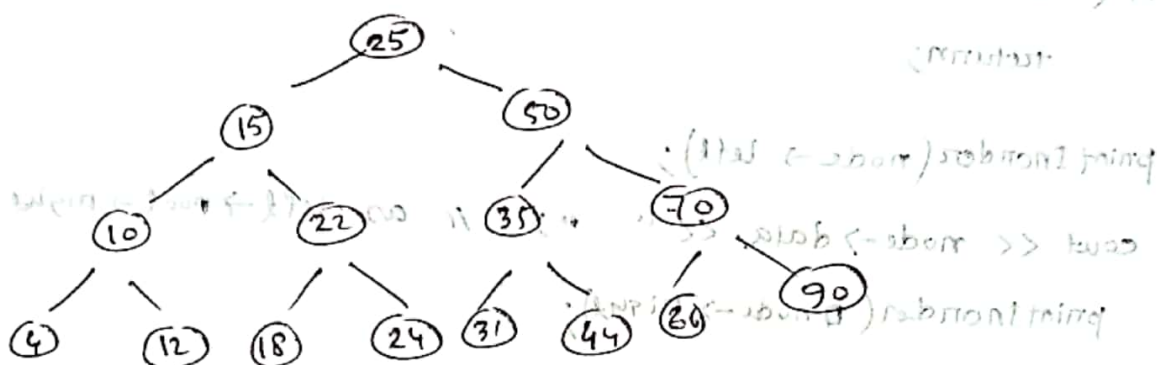
```
root->right = SortedArrayToBST(arr, mid+1, end);
```

```
return root;
```

```
}
```

Ques-05

Tree Output:



In-Order Traversal:

4 10 12 15 18 22 24 25 31 35 44 50 66 70 90

Pre-Order Traversal:

(Root → left → right)

25 15 10 22 4 12 18 24 50 35 31 44 70 66 90

Post-Order Traversal:

↓

(left → right → Root)

4 12 31 44 35 66 90 70 50 18 24 22 10 15 25

Tree Traversal

```
void printInorder ( struct node* node)
```

```
{ if (node == NULL)
    return;
```

```
printInorder (node->left);
```

```
cout << node->data << " ";
```

```
printInorder (node->right);
```

```
}
```

```
void preOrder (struct node* node)
```

```
{ if (node == NULL)
```

```
    return;
```

```
cout << node->data << " ";
```

```
preOrder (node->left);
```

```
preOrder (node->right);
```

```
}
```

```
void postOrder (struct node* node)
```

```
{ if (node == NULL)
```

```
    return;
```

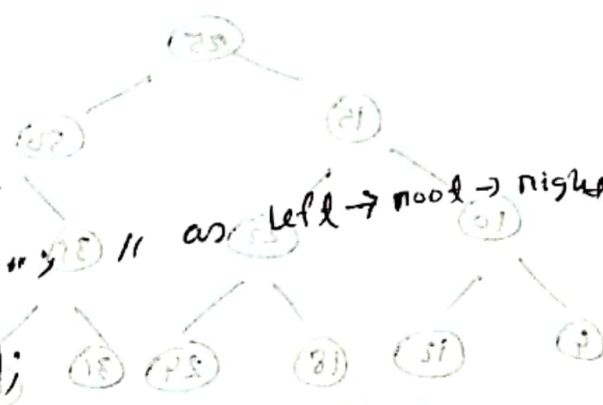
```
postOrder (node->left);
```

```
postOrder (node->right);
```

```
cout << node->data << " ";
```

```
}
```

Tree Traversal



node->left->right

(left->right->left)

node->right->left

(right->left->right)